

MASTER REFERENCE / VERSION 2.0

ApplySync

Privacy-first Gmail job application tracker

Architecture, six-stage delivery plan and AI-assisted learning workflow

Agent implements. Sanjay directs, investigates and verifies.
You do not need to memorize or type every line. You need to find the relevant code, explain it, change it and check whether the change works.

Document control	Value
Version / date	2.0 / 17 September 2026
Owner / project	Sanjay Kumar / ApplySync
Authority	Consolidates Master v1.0 and Errata v1.1; replaces their old teaching schedule with six delivery stages.
Delivery status	Health endpoint completed, as reported by Sanjay. Other capabilities in this document are planned requirements.
Repository	https://github.com/sanjaykumarmugada18/applysync.git
Timing	Demo deadline and daily project hours are not yet confirmed. No completion-date promise is made.

The product idea, approved stack and privacy/correctness requirements remain intact. This revision changes how implementation and learning are organized. Existing source PDFs remain historical records.

NAVIGATION

Contents

- 03** Authority and current state
- 04** Product and system workflow
- 05** Approved technology stack
- 06** Roles and learning method
- 07** Codex-to-chat review workflow
- 08** Authentication and security
- 09** Privacy and classification
- 10** Gmail connection and setup
- 11** Synchronization and recovery
- 12** Data model and application events
- 13** Application matching policy
- 14** Thread bindings and edge cases
- 15** Build 1 - Backend & Persistence
- 16** Build 2 - Frontend & Integration
- 17** Build 3 - Authentication & Ownership
- 18** Build 4 - Gmail & Reliable Synchronization
- 19** Build 5 - Classification & Human Review
- 20** Build 6 - Automation & Release
- 21** Verification and completion
- 22** Open decisions, sources and change record

Read according to the task

Start with pages 3, 6 and 7 for authority and working agreements. Read the current delivery-stage page before asking Codex to implement. Consult the architecture and security pages for the feature being built. The reference describes the full target; it does not authorize building future stages in advance.

01 / GOVERNANCE

Authority and current state

One current reference

This v2.0 document is the consolidated project baseline. It incorporates the architectural corrections from Errata v1.1 and the workflow changes explicitly agreed with Sanjay on 17 September 2026. Routine work should not require reconciling the two older PDFs.

- Keep Master v1.0 and Errata v1.1 as archived history. Their 25-phase teaching sequence and Phase-1-only starting instructions are superseded here.
- Use six sequential delivery stages. Subdivide each into bounded, reviewable tasks. Do not generate the whole repository or all stages in one pass.
- Codex may write substantial implementations, configuration, migrations and tests. Manual first attempts, retyping and a prescribed percentage of handwritten code are not required.
- Preserve correctness, authorization, privacy, conservative matching and verification. A tighter deadline changes task organization; it does not silently weaken safeguards.

Verified versus reported progress

Item	State from the supplied handoff
Local setup	Windows, VS Code, PowerShell; C:\Users\sanja\Desktop\ApplySync; .venv; FastAPI and Uvicorn.
Implemented	main.py and .gitignore. GET /health returns {"status":"healthy"}. Missing route returns 404. Restart after laptop shutdown succeeds.
Learning evidence	Sanjay explained request flow and temporarily changed the JSON response, then restored it.
Git	Reported local commit 37019df: Build first FastAPI health endpoint. Remote push and current working-tree status need inspection.
Next work	Build 1, continuing from the existing endpoint. PostgreSQL setup and all later features remain unverified.

This document is not a repository audit. Do not claim that planned features exist or that tests passed without inspecting the actual code and output. Original Phase 1 is complete; the old Phase 2 teaching prompt is superseded, not a reason to restart.

Applying the change

After adding this reference to Project resources, update the Project custom instructions and repository AGENTS.md to agree with it. Those settings and files are separate actions; generating this PDF does not modify them. Record later architecture corrections with rationale, affected behaviour and tests.

02 / PURPOSE AND ARCHITECTURE

Product and system workflow

ApplySync tracks hiring-process progress from job-related Gmail messages. It gives a busy candidate a dashboard of applications, stages and event history, while preserving human review for uncertain or contradictory evidence. Manual application entry provides an early usable workflow and remains a supported input.

Product promise

The dashboard updates automatically after the next scheduled Gmail synchronization. Uncertain matches and classifications require review. Do not promise guaranteed real-time updates or perfect extraction.

Many common tracking workflows require candidates to save opportunities and maintain stages manually. ApplySync differentiates through dedicated label-scoped ingestion, metadata-first processing, application matching, event history, transition checks and no long-term raw-email storage. Do not claim that all competitors or general-purpose tools lack Gmail or AI automation.

End-to-end flow

- A user registers or logs in to ApplySync. The application establishes its own cookie-based session.
- The user separately authorizes Gmail access. Guided onboarding verifies the JobTracker label and required bounded filter groups.
- A bounded historical backfill establishes a baseline. Incremental History API polling later discovers relevant changes.
- GmailConnector retrieves only eligible labelled messages, starting with metadata. The ingestion ledger records durable progress.
- Privacy checks precede deterministic rules. Only safe, still-ambiguous text is sent to Groq for a validated structured proposal.
- Backend policy resolves application identity, checks transitions and records events. Ambiguity goes to review rather than a guess.
- One reducer derives the current application stage. The frontend displays results and allows auditable corrections and merges.
- Background workers automate the proven sync/classification pipeline. Failures remain observable and retryable.

Two different uses of AI

Development: Codex helps create and test repository code under the task constraints. **Inside ApplySync:** the classifier is a non-agentic model call with no Gmail, database, browser or email-sending tools. Using a coding agent does not turn the product into an autonomous email agent.

“Apply” identifies job applications; “Sync” identifies scheduled synchronization. Interview explanations must distinguish implemented behaviour from this target design.

03 / STACK AND BOUNDARIES

Approved technology stack

Area	Approved components and purpose
Backend	Python, FastAPI for HTTP APIs; Pydantic for request/response and structured-output validation.
Persistence	PostgreSQL; SQLAlchemy models, queries and sessions; Alembic schema migrations.
Frontend foundation	Vite + React + TypeScript; React Router; basic JavaScript and TypeScript taught in the current feature. No Next.js substitution.
Frontend behaviour	Plain fetch for understanding transport; TanStack Query for server state; React Hook Form and Zod for forms/validation.
Interface	Tailwind CSS for responsive accessible styling; @dnd-kit for auditable stage movement in Build 5.
Verification	Pytest; Vitest and React Testing Library; Playwright for important end-to-end flows. Introduce checks with relevant behaviour.
Gmail	Google OAuth authorization-code flow; gmail.readonly; user-created JobTracker label and bounded filters; Gmail History API.
Classification	Deterministic rules first; Groq structured classification for eligible ambiguous messages; labelled evaluation dataset and human review.
Jobs	Redis and a background worker. Exact worker library chosen in Build 6; not silently assumed to be Celery or RQ.
Release	Git/GitHub; Docker/Compose; GitHub Actions; HTTPS; production PostgreSQL; structured logs, health checks and error monitoring.

Choices should solve the current requirement

Use the approved stack, with compatible dependency versions recorded when installed. A new stage permits necessary structure; it does not justify unused abstractions, empty future modules or speculative infrastructure. The backend can adopt simple responsibility-based files during Build 1 rather than waiting for a separate refactoring lesson.

Briefly inspect a plain-fetch request before using TanStack Query. Rebuilding the whole interface twice is not required. Detailed folder layouts, provider versions and deployment settings are implementation decisions to document when needed, not facts invented by this reference.

04 / AGENT-ASSISTED ENGINEERING

Roles and learning method

Working cycle: understand the outcome, specify a bounded task, let Codex implement, inspect the result, verify behaviour, explain the important flow, then commit a working checkpoint.

Participant	Responsibility
Sanjay	Understand the problem and acceptance criteria; inspect meaningful behaviour; ask questions; participate in debugging; practise focused changes and explain decisions honestly.
Codex in VS Code	Inspect repository state; implement the scoped change; run relevant checks; explain changed files and important logic; report assumptions, failures and unrun checks.
Stage chat mentor	Explain unfamiliar concepts, refine tasks, review supplied code and evidence, examine architectural choices, and help identify understanding gaps.
Planning chat	Maintain stage status, scope decisions and handoffs. Keep the active reference and instructions consistent.

Understanding at three depths

- **System:** identify the components and trace data between them. Know why each component exists.
- **Feature:** locate the real endpoint, schema, service/query and response; follow one successful and one failing execution path.
- **Critical logic:** closely inspect ownership filters, transactions, checkpoint updates, privacy gates, state transitions and matching conditions.

Do not require line-by-line narration of every file, repeated explanations of familiar imports, mastery of library internals or memorized syntax. If unfamiliar logic prevents evaluation, address that gap before adding more complexity.

Targeted practice, not retyping

Use small demonstrations, skeletons or hints where helpful, without making them mandatory prerequisites for every feature. At meaningful checkpoints, Sanjay makes a small behavioural change or investigates a failure with documentation and focused help. The agent may generate most code. Independent DSA practice remains separate from repository authorship.

Bound tasks by reviewability

Agree on goal, allowed scope, exclusions, acceptance criteria and checks. Ask for a short approach before substantial changes. Routine steps within the agreed task can proceed; architectural departures require discussion. Stop the task when its result is reviewable, not after unrelated “improvements.”

05 / EVIDENCE AND SHARED UNDERSTANDING

Codex-to-chat review workflow

Do not assume the stage chat can see VS Code, local files, terminals, the Codex conversation or private GitHub content. Conversely, do not assume Codex sees ChatGPT Project resources. Supply the relevant reference and working instructions in each environment.

A review package after a meaningful change

Include	Required detail
Identity	Build stage, task, branch, base commit and current commit if committed; state whether work remains uncommitted.
Files	Changed file names with each responsibility; relevant diff plus complete new files and necessary context.
Flow	Entry point, validation, business logic, persistence/external calls, response; one failure path.
Evidence	Exact checks run with observed results; explicitly label checks not run, failed or blocked.
Decisions	New dependencies, schema/configuration changes, assumptions, limitations and unresolved questions.
Learning	Two or three important concepts and what Sanjay should inspect or try next.

How Sanjay shares it

For small changes, paste the package with relevant code. For larger changes, attach the changed source files and tests. Screenshots help with UI appearance; code and terminal text are better for implementation review. A reachable commit or pull-request link can identify a version, but a private URL alone does not grant access.

Use VS Code Source Control to inspect changed files and diffs. A diff shows additions and removals. Ordinary git diff does not include untracked new files, and staged changes are separate; ensure the package includes both. Do not upload .venv, dependency directories, .env contents, credentials or real email content.

Reusable request to Codex

Prepare a review handoff for this task. List changed files and responsibilities; include the relevant diff and new files; trace one success and one failure path; report exact checks and results, unrun checks, assumptions and Git state. Explain the important unfamiliar logic using actual file and function names. Do not include secrets or sensitive data.

How the mentor reviews

Start with file responsibilities and one execution path, then inspect important lines and test evidence. Separate “Codex reported” from “code inspected” and “check independently run.” A summary is enough for progress tracking; it is not enough to certify correctness. Resolve concrete gaps without duplicating every check or requiring a mentor round-trip for every edit.

06 / IDENTITY AND LEAST PRIVILEGE

Authentication and security

Application identity is separate from Gmail authorization

- Register/login/logout/me endpoints establish an ApplySync identity. Hash passwords using an established password-hashing library; do not store plaintext or reversibly encrypted passwords.
- Store the application JWT in an HttpOnly cookie. Use Secure in HTTPS production and choose SameSite deliberately for the deployment topology.
- Protect state-changing requests against CSRF where the topology requires it; configure credentialed CORS narrowly. CORS alone is not an authorization control.
- Derive ownership from the authenticated session. Scope every relevant data operation to that user, including direct API requests, jobs, review, merges and Gmail connection access. Non-guessable IDs do not replace ownership checks.
- Never put Gmail access/refresh tokens in the application JWT, API responses or React. Test expiration, logout, missing sessions, cross-site behaviour and missing/invalid CSRF protection.

GmailConnector boundary

One internal package centralizes refresh-token decryption, authenticated Gmail client creation, Gmail API access and label verification. Other code calls narrow named operations such as `verify_configuration` or `list_labelled_message_ids`. Never expose raw credentials, a generic Gmail client or an unrestricted Gmail-request function.

Use architecture tests based on AST/import-boundary inspection to reject Gmail client imports, token decryption, client construction and direct endpoint references outside the approved package. Run these in CI once CI is introduced.

This is an architectural least-privilege boundary, not a runtime sandbox. Arbitrary code execution or a deliberate bypass inside the same process could still access secrets. Tests reduce accidental boundary violations; they cannot provide process or IAM isolation.

Threat controls retained from the original design

Threat	Required mitigation
Token theft / database access	Encrypted refresh tokens, memory-only access tokens, revocation, minimal stored data and least-privilege database access.
Cross-user access / insider access	Ownership tests; controlled administrative access; no routine ad hoc access to users' production data.
Logs / backups	Allowlisted structured logs; sanitized error reporting; encrypted backups and disclosed deletion/retention handling.

07 / MINIMIZE DATA BEFORE INFERENCE

Privacy and classification

Google grants read-only mailbox access. ApplySync uses it for limited, label-scoped ingestion of likely job-application messages and does not store raw email bodies or attachments. The label is a data-minimization control, not an OAuth-level restriction.

Processing sequence

Eligible message ID → metadata checks → sensitive-category block → body retrieval only when justified → MIME parsing and HTML-to-text → remove quoted history/signatures → redact → deterministic rules → eligible ambiguous excerpt to Groq → validated structured facts → discard raw content.

- Check sensitive categories before any classifier. Use metadata first and further checks when body text is available; do not assume a label proves an email is safe. Block OTPs, banking, tax, medical, legal, password resets, shopping receipts and security alerts.
- Retrieve only the eligible individual message. Do not fetch complete threads or attachments. Do not load remote images or tracking pixels.
- Raw email content is temporary processing data. Never persist bodies/HTML in PostgreSQL, job payloads or logs. Show validated structured plain-text fields in the frontend, not raw email HTML.
- Logs must not contain tokens, subjects, bodies, attachments, model prompts or raw model responses. Preserve original observed company names only as justified structured data, not leaked email content.

Rules first; model proposals second

Classify obvious confirmations and interview messages deterministically. Groq receives only the minimal redacted excerpt needed for safe ambiguous cases. Sending it data creates another recipient: verify and disclose the applicable provider retention/data policy before use.

Treat email as untrusted data, never instructions. The model has no tools or database authority. Validate strict structured output; reject malformed output and unsupported values. Schema validation limits accepted structure but does not establish factual correctness or eliminate semantic prompt injection.

Backend policy accepts, reviews or rejects proposals. Keep event-classification confidence separate from application-identity confidence. A model confidence number is not a calibrated probability and never alone authorizes an update.

Failure and evaluation

Model failure must not erase the ingestion ledger. Maintain retryable classification state separately from sync state. Evaluate a small labelled dataset for accuracy, false positives and malformed outputs. Privacy tests prove that sensitive fixtures never invoke the mocked Groq client. Use synthetic/redacted fixtures, not personal emails sent to coding assistants.

08 / OAUTH AND ONBOARDING

Gmail connection and setup

Token and OAuth-state handling

- Request the approved gmail.readonly scope. No Gmail send, compose, modify or delete scopes; no automatic filter/label creation in v1.
- Request offline access. A later OAuth response may omit a refresh token: encrypt/store one when returned and never overwrite an existing valid token with null.
- Force re-consent only when genuinely necessary. Explicitly handle missing, revoked, expired and reissued refresh-token cases. Access tokens remain memory-only.
- Generate secure state, bind it to the current user/session, make it single-use and reject missing, modified, expired or replayed state. Validate connection ownership.
- Disconnect revokes authorization and removes token material. Verify the disconnect flow, failure behaviour and UI state without exposing credentials.

User-created setup

Guide the user through creating JobTracker and multiple bounded filters. Resolve the label's immutable ID. Use the narrow connector operation to inspect filters and verify coverage; one filter adding the label is not sufficient evidence of complete setup.

Coverage group	Onboarding display and role
JobTracker label	Configured/missing; required data-minimization mechanism.
ATS filters	Coverage of applicant-tracking-system senders.
Assessment/scheduling filters	Coverage of assessment and scheduling providers.
Strong-subject filters	Coverage of specific job-process subject phrases.
Employer-specific filters	Optional coverage for employer domains.

Define the minimum required filter-group coverage explicitly in Build 4, test it and block synchronization until it passes. Employer-specific filters remain optional. Narrow sender rules may miss recruiter replies; broad keywords may ingest unrelated mail. Multiple bounded groups manage this trade-off without claiming perfect coverage.

External requirements

Verify current Google scope/method compatibility and applicable OAuth consent/verification requirements when implementing and before public release. The source design uses filters.list under gmail.readonly; do not silently broaden permissions if an integration check fails. Record a concrete conflict for review.

A successfully connected account is not permission to scan the entire inbox. Verify current label eligibility before treating retrieved content as an ingestion candidate.

09 / DURABLE INGESTION

Synchronization and recovery

Baseline and incremental flow

Start with a manual Initial Sync action. Backfill only labelled messages within a bounded date range, using pagination and metadata first. Exclude spam, trash and unlabelled candidates. Establish a processed-message ledger before consuming incremental changes.

Store the Gmail historyId checkpoint. Process relevant messageAdded and labelAdded events, deduplicate appearances across change types, and verify the required label is still present. A label applied after delivery must still make the message discoverable.

Mandatory invariants

- Enforce UNIQUE(gmail_connection_id, message_fingerprint) at database level. Use HMAC-based fingerprints for stable matching; randomized ciphertext is not a reliable equality/uniqueness key.
- Serialize synchronization per Gmail connection. Only one operation may advance its checkpoint at a time. Application-level “already exists” checks alone do not prevent races.
- Advance historyId only after the relevant ingestion changes are successfully and durably handled. Never advance past a failed ingestion record.
- Keep synchronization state and classification state separate. Successful durable ingestion can preserve pending/retryable classification without waiting for a successful LLM call; an LLM failure must not lose the message ledger.
- Use transactions and retry-safe processing. At-least-once execution must not create duplicate ledger entries, application events or thread bindings.
- An expired-history 404 triggers bounded label-scoped reconciliation, not an unrestricted inbox scan. Record sync-run outcomes and retryable failures without email content.

How to reason about a failure

Scenario	Required outcome
Same message appears twice	One ledger record under the database constraint.
Ingestion fails partway	No checkpoint skips unhandled changes; retry remains safe.
Two sync requests overlap	Per-connection serialization protects progress.
Classification fails	Ledger survives; classification can be retried independently.
History has expired	Reconcile the permitted label scope and establish safe new progress.

Automate only after the manual pipeline is correct. Build 6 introduces polling, workers and visible job status. Polling frequency, locks and transaction boundaries must be chosen and documented from concrete requirements; this reference does not pretend those implementation details are already settled.

10 / IDENTITY, FACTS AND DERIVED STATE

Data model and application events

Entity	Responsibility
User / Gmail connection	Application ownership and authorized external connection; introduced only in their relevant stages.
JobApplication	One hiring-process application; owns event history, thread links and external identifiers. Company + role is not a unique identity.
ApplicationEvent	Immutable structured fact or manual action: confirmation, assessment, interview, rejection, offer or correction.
ProcessedGmailMessage	Durable ingestion record, unique within its connection; may link to an application and produce an event. Sync/classification states stay separate.
ApplicationThreadLink	Connection-scoped thread binding, link source and evidence/score; preserves superseded inactive links.
ApplicationExternalIdentifier	Issuer/employer-scoped requisition IDs, ATS application IDs and exact job URLs for strong identity evidence.
Sync/job audit records	Track progress, failures and recovery without persisting sensitive content.

One source for the current stage

Once the event model is introduced in Build 5, derive the current stage through one central `derive_application_state(events)` reducer with explicit valid transitions. Do not scatter independent status assignments across routes, background jobs and UI handlers.

- A manual stage change creates an event. Drag-and-drop invokes that domain operation; it does not directly overwrite the authoritative stage.
- Corrections supersede incorrect events rather than deleting history. Preserve enough lineage to explain what changed and why.
- A merge reassigns valid links and related data transactionally while preserving auditability. Review decisions belong to the authenticated user.

Progressive construction

Build 1 needs only the initial application model and persistence/API requirements. Do not create all these entities in advance. When introducing events, migrate existing manual records coherently and remove competing status-update paths. The final event invariant does not require building the event system in the first task.

Schema discipline

Use Alembic migrations, inspect generated changes and verify the real database. Introduce indexes and constraints for demonstrated invariants. Ownership is enforced by server-side identity and queries, not by frontend hiding or trusting a supplied user ID. Detailed field names, enum values and migration choices are recorded with the implementation.

11 / CORRECTED V1 DECISION RULE

Application matching policy

Message deduplication and application identity are different problems. Two different emails can concern one application; two applications can share a company and role. Never merge solely because a company, role or sender domain matches.

Evidence order and existing bindings

Consider valid scoped external identifiers, an existing confirmed thread binding, exact job/requisition URLs and then independent cross-thread evidence. A thread binding is a useful established relationship, not an irreversible truth. Strong contradictory evidence or multiple plausible applications requires review.

Creating an automatic binding: rule A OR rule B

A. Exact identifier rule: exactly one application has a valid, properly scoped exact identifier match: an ATS application ID, employer/issuer-scoped requisition ID, exact job URL or other unique hiring-process reference. A requisition number alone is not globally unique.

B. Fixed multi-signal rule: every condition below must hold:

- The top candidate has at least three independently agreeing positive signals.
- One signal is a canonical-company match.
- At least one signal is hiring-process-specific: normalized role, recruiter relationship, job URL, assessment context, or application timeframe plus a valid transition.
- There is no strong contradiction.
- The top candidate exceeds the runner-up by at least two independent signals.
- Exactly one candidate satisfies the complete rule.

Otherwise: manual review. A two-signal advantage alone is insufficient. Do not count duplicated representations of the same fact as independent evidence. Resolve signal definitions and scoring examples with tests during Build 5.

Creation and classification limits

An explicit application confirmation can create a new application and bind its thread when the creation decision is justified. A later-stage interview, offer or rejection with no reliable match must not silently create a new application. A confident event classification does not establish application identity. Messages mentioning multiple applications require review.

This conservative fixed rule is the v1 policy. Statistically calibrated matching thresholds remain deferred until evidence from a labelled dataset and actual usage justifies the change.

12 / AUDITABLE IDENTITY DECISIONS

Thread bindings and edge cases

All six binding sources

Source	When a binding is written
AUTO_APPLICATION_CREATION	An explicit confirmation creates a new application; bind its thread immediately.
AUTO_EXTERNAL_IDENTIFIER	A valid scoped identifier links a new thread to exactly one existing application.
AUTO_STRONG_MATCH	The complete corrected multi-signal rule passes for exactly one candidate.
MANUAL_REVIEW	The user selects the correct application for ambiguous evidence.
MANUAL_CORRECTION	The user reassigns a message; supersede the wrong link and preserve its history.
MERGED_APPLICATION	Merge duplicates and reassign valid links in one transaction.

Uniqueness without losing history

Only one active binding may exist for a Gmail connection and thread. Use a partial unique index over (gmail_connection_id, thread_fingerprint) WHERE is_active = true. A plain uniqueness constraint across active and inactive rows would prevent preserving historical links.

Required edge cases

- Same company with two concurrent roles: company alone cannot select a candidate. Require strong identity evidence or user review.
- Same company and role in separate cycles: do not enforce company+role uniqueness. A new explicit confirmation may represent a genuine reapplication.
- Employer legal-name variants: canonicalize identity while retaining the observed structured name where needed. Normalization does not by itself prove application identity.
- ATS sender versus employer: the sender domain can identify a platform, not the hiring company. Use message facts and existing context.
- New recruiter thread for an existing application: evaluate cross-thread matching. Thread ID alone cannot join it.
- Contradiction to an existing binding, multiple plausible candidates or multiple applications in one message: request review instead of trusting the old link or guessing.

Keep link source, active/inactive history and relevant structured evidence explainable. Do not retain raw email bodies as an audit shortcut.

13 / DELIVERY STAGE 1 OF 6

Build 1 - Backend & Persistence

Working result: create, list and retrieve manual applications in PostgreSQL through a tested API. Continue from the existing health endpoint; do not rebuild it.

Bounded implementation sequence

- **Inspect:** check the actual repository, branch, working tree, dependencies and local PostgreSQL availability. Preserve existing work and unrelated databases.
- **Persistence:** introduce local configuration, secret handling, SQLAlchemy connection/model and the first Alembic migration. Inspect the table directly and confirm applying upgrade head again is safe.
- **API:** implement POST /applications, GET /applications and GET /applications/{id}, with Pydantic request/response schemas and appropriate status/error behaviour.
- **Structure and checks:** use simple files for demonstrated responsibilities and add Pytest coverage alongside the behaviour. No requirement to write a monolith first and refactor later.

What Codex may implement

Connection setup, initial model, migrations, schemas, endpoints, fixtures, tests and setup documentation. Record dependencies and configuration requirements. Keep secrets in local environment configuration and out of Git. Do not silently use `create_all()` instead of the required migration workflow.

What Sanjay learns and verifies

Trace request → validation → session/query → transaction → database → response. Distinguish Python model, Alembic migration and actual table. Inspect an actual saved record, invalid input and an unknown ID. Attempt one useful validation change or investigate a persistence failure with focused help.

Exit evidence

- Health still works; create/list/detail work; data survives restart; invalid input and missing IDs behave as specified.
- Migrations apply correctly; relevant tests pass with isolated test data and no order dependence. A meaningful broken behaviour is caught by a corresponding check.
- Provide a review package, explain the important flow and commit a working result. Report pushed versus local-only state honestly.

Scope boundary

No React, Gmail, model calls, workers or future entities. Authentication arrives in Build 3. Early unauthenticated development is a local learning/build state, not a multi-user production release.

Chat: Build 1 - Backend & Persistence. Historical coverage: original phases 1-5; original phase 1 already complete.

14 / DELIVERY STAGE 2 OF 6

Build 2 - Frontend & Integration

Working result: a usable browser interface that creates and displays real applications, with clear loading, empty, validation and failure states.

Bounded implementation sequence

- **Display:** Vite, React and TypeScript; reusable application cards, stage badges and a dashboard summary using sample data.
- **Interaction:** React Router pages for dashboard, list, detail and new application; forms, local state and visible validation.
- **Integration:** inspect a plain-fetch request and connect the actual backend. Explain JSON, async flow, environment configuration and CORS.
- **Server state:** use TanStack Query for queries, mutations and invalidation. A small fetch demonstration is sufficient; do not rebuild the whole app solely to repeat the lesson.
- **Usability:** Tailwind styling, responsive layout, visible input labels and keyboard focus. Introduce React Hook Form/Zod where the form requires them.

What Codex may implement

Component files, routing, types, forms, API integration, styling and relevant component/integration checks. Generate the normal implementation directly, then explain representative components and the important request path.

What Sanjay learns and verifies

Learn JavaScript arrays/objects, map, destructuring and modules as used. Understand JSX, props, keys, events, state, immutable updates and basic TypeScript types. Trace submit → request → backend response → refreshed screen. Distinguish local client state from persistent server data; advanced generics are not a prerequisite.

Exit evidence

- Create an application through the UI; retrieve it from the database; reload the browser and see persisted data.
- Creating a record refreshes the list appropriately. Invalid input and network/server failure show clear feedback; no silent success.
- Responsive views, accessible labels and keyboard focus work. Relevant frontend checks pass; retain backend checks.
- Sanjay explains one component and one request flow, and investigates or changes a small behaviour. Commit the working result with review evidence.

Scope boundary

No Gmail or classification. Authentication comes next; drag-and-drop must wait for auditable backend stage-change operations in Build 5. No separate full styling rewrite or framework substitution.

Chat: Build 2 - Frontend & Integration. Historical coverage: original phases 6-10.

15 / DELIVERY STAGE 3 OF 6

Build 3 - Authentication & Ownership

Working result: each user can register, log in, restore a session and log out; application data is protected by backend ownership checks.

Bounded implementation sequence

- **Backend identity:** /auth/register, /auth/login, /auth/logout and /auth/me; password hashing, application JWT cookie and current-user dependency.
- **Ownership:** introduce user ownership with an explicit migration strategy for earlier local application records. Scope every application query and mutation.
- **Browser session:** register/login pages, protected navigation, session restoration, credentialed requests and logout behaviour.
- **Security verification:** test unauthenticated access, expired sessions, cross-user requests, cookie policy, CORS and required CSRF defences.

What Codex may implement

Models/migrations, established authentication library integration, endpoints, dependencies, frontend auth state and security tests. Explain the cookie/session flow and each important ownership condition. Do not invent custom cryptography.

What Sanjay learns and verifies

Explain authentication versus authorization; hashing versus encryption; HttpOnly versus Secure; 401 versus 403; and why a protected frontend route cannot secure an API. Locate the query condition that prevents User A reading User B's record. Follow logout and expired-session behaviour in the actual browser/API flow.

Exit evidence

- Register/login/profile/logout work through the UI and API. Session restoration across refresh behaves as specified.
- Two-user tests prove direct modified-ID requests cannot disclose or change another user's data. Unauthenticated calls fail as specified.
- No Gmail data is present in application JWTs. Secrets are not exposed in logs, frontend bundles or committed configuration.
- CSRF and cross-site tests match the chosen topology. Relevant security and existing feature checks pass; commit and summarize the working state.

Scope boundary

This establishes ApplySync identity. Gmail OAuth is a separate authorization relationship in Build 4. Do not conflate the two flows or add Gmail fields merely because login now exists.

Chat: Build 3 - Authentication & Ownership. Historical coverage: original phases 11-12.

16 / DELIVERY STAGE 4 OF 6

Build 4 - Gmail & Reliable Synchronization

Working result: connect Gmail and reliably ingest permitted message metadata through a manually triggered, retry-safe synchronization flow.

Bounded implementation sequence

- **Connection:** introduce GmailConnector, Connect Gmail, callback/state validation, encrypted refresh-token handling and disconnect. Initially fetch no messages.
- **Setup verification:** resolve JobTracker label ID, inspect filter-group coverage, define the minimum setup and enforce the connector boundary with architecture tests.
- **Baseline:** bounded historical backfill, pagination, metadata-first retrieval and a database-unique HMAC fingerprint ledger.
- **Incremental path:** historyId, messageAdded + labelAdded, current label checks and duplicate-change handling.
- **Recovery:** per-connection serialization, safe checkpoint advancement, transactions, retries, expired-history reconciliation and sync-run records.

What Codex may implement

Connector operations, OAuth handlers, storage/migrations, setup UI, sync services and mocked integration/security tests. Verify current official Gmail behaviour before relying on external API details. Keep credentials and real email contents out of review handoffs.

What Sanjay learns and verifies

Trace authorization-code exchange and token use without revealing tokens. Explain why label restriction is enforced by the application, why a refresh token may be absent on reauthorization, how checkpoints work, and why retries cannot create duplicates. Diagnose one mocked sync failure.

Exit evidence

- Connect/disconnect work; missing, modified, expired and replayed state is rejected. Omitted refresh tokens do not erase valid stored tokens; ownership is enforced.
- Invalid setup blocks sync. Connector tests catch forbidden access paths. No full-thread or attachment retrieval occurs.
- Backfill/incremental checks cover duplicate messages, labels added after delivery, failed ingestion, concurrent syncs and expired-history recovery.
- Safe logs and durable ledger state support retry. Review package identifies passed, mocked and not-yet-live checks; commit working code.

Scope boundary

No body classification or live LLM calls in this stage. Background scheduling is deferred to Build 6 so the manual pipeline can be understood first.

Chat: Build 4 - Gmail & Reliable Synchronization. Historical coverage: original phases 13-17.

17 / DELIVERY STAGE 5 OF 6

Build 5 - Classification & Human Review

Working result: eligible messages become validated, correctly matched application events; uncertain cases and corrections are handled through an auditable review interface.

Bounded implementation sequence

- **Privacy/rules:** metadata/body-on-demand checks, sensitive-category blocks, MIME handling, text cleanup, redaction and deterministic classification.
- **Structured model:** Groq for safe ambiguity, strict schema validation, failure/retry handling and a small labelled evaluation dataset.
- **Events:** application timeline, explicit transitions, one reducer and correction/supersession semantics; reconcile existing manual records.
- **Identity:** corrected identifier/multi-signal policy, thread bindings, all binding sources, uniqueness and review fallback.
- **Human controls:** classification review, application selection, manual corrections, transactional merges and @dnd-kit stage movement backed by events.

What Codex may implement

Parsers, privacy checks, rule/model integration, schemas, reducers, matching services, migrations, UI and focused tests. Keep each subtask reviewable; do not generate the entire classification/matching subsystem in a single opaque change.

What Sanjay learns and verifies

Trace a synthetic message through privacy screening, classification, identity resolution and state derivation. Explain why valid JSON can still contain incorrect facts, why company similarity is not identity, and why a manual correction needs history. Inspect one wrong-match or invalid-transition case.

Exit evidence

- Sensitive fixtures never call Groq; quoted history and remote content are handled as required; raw content is not persisted or logged.
- Malformed output is rejected; failed classification remains retryable without losing ingestion records; evaluation metrics and limitations are reported.
- Matching tests cover confirmations/interviews joining correctly, two roles staying separate, reapplications, ambiguity and contradictory bindings.
- Only one active binding exists per thread; inactive history is preserved. Review, correction, merge and drag actions are authorized and auditable.
- Stage derives from the central reducer; backend/frontend tests pass; Sanjay explains the critical decisions and commits the reviewed result.

Chat: Build 5 - Classification & Human Review. Historical coverage: original phases 18-22.

18 / DELIVERY STAGE 6 OF 6

Build 6 - Automation & Release

Working result: a reproducibly deployed system that runs scheduled sync, reports failures safely and supports tested recovery, disconnect and deletion.

Bounded implementation sequence

- **Automation:** choose the worker library, add Redis, scheduled History API polling, classification jobs, retries/backoff and visible job status.
- **Correct execution:** preserve per-connection serialization and idempotency across workers. Long-running tasks must not hold HTTP requests open.
- **Containers:** Dockerfiles and Compose for frontend/backend/worker/PostgreSQL/Redis with needed networks, volumes and health checks.
- **CI:** GitHub Actions runs backend, frontend, ownership, OAuth, privacy, matching and architecture-boundary checks. Use mocked external services appropriately.
- **Production:** deploy with HTTPS, protected environment configuration, safe migration execution, structured logs, health checks, error monitoring and backup/restore procedures.

What Codex may implement

Jobs, configuration, deployment artifacts, CI and operational documentation within the approved environment. Record exact setup commands and distinguish local, test and production settings. Public deployment, provider configuration and external approvals must be verified rather than assumed complete.

What Sanjay learns and verifies

Explain queue versus worker versus scheduler; retries and idempotency; image versus container; persistent volumes; secret configuration; and why a failed sync can be diagnosed without logging email content. Practise an operational failure with safe test data.

Exit evidence

- Scheduled sync runs without blocking requests. Retries and overlapping workers do not duplicate data or corrupt progress; failed jobs are visible and safely retryable.
- A clean setup can start the documented stack. CI catches a deliberate connector-boundary violation and relevant behaviour regressions.
- Real HTTPS deployment works; production Gmail connection/disconnection is tested; health and monitoring are useful without leaking content.
- Disconnect and delete-all behave as specified; backup retention/deletion implications and known limitations are disclosed. Commit the reproducible release configuration.

Deferred v2 scope

Keep Pub/Sub push notifications, a separately deployed token/connector service, automatic duplicate merging, statistically calibrated matching and automatic Gmail filter/label creation deferred. Polling remains the v1 mechanism; do not promise real-time delivery.

Chat: Build 6 - Automation & Release. Historical coverage: original phases 23-25.

19 / EVIDENCE BEFORE PROGRESS CLAIMS

Verification and completion

Testing accompanies the feature

Codex may generate tests alongside implementation; there is no need to wait for an old numbered phase. Match checks to concrete risks, inspect important assertions, and use external-service mocks in routine tests. A model-generated “all good” summary is not independent verification.

Area	Minimum meaningful evidence
API and persistence	Valid operation, invalid input, missing record, restart persistence, migration correctness and test-data isolation.
Frontend	Submission/rendering behaviour, loading/empty/error states, accessible labels and keyboard focus; important integrated flows.
Identity	Unauthenticated and expired-session requests; two-user isolation; cookie/CORS/CSRF behaviour for the actual topology.
OAuth/Gmail	Invalid and replayed state, refresh-token omission, label/filter gating, connector boundary, forbidden retrieval and disconnect.
Sync	Duplicate changes, concurrency, checkpoint failure, retry, expired history and separate classification state.
Classification/matching	Sensitive-data non-invocation, malformed output, injection cases, evaluation results, ambiguous identity and matching edge cases.
Release	Worker retries, reproducible setup, CI, safe production observability, disconnect/delete-all and documented retention behaviour.

A stage is complete when

- Its intended behaviour works and the relevant success/failure checks pass. Important limitations and unrun checks are explicit.
- Sanjay can locate and explain the important execution path and has made a meaningful small change or participated in a concrete diagnosis. This is not an unaided rewrite of generated code.
- Critical logic and tests have been reviewed at the appropriate depth. A code review claim identifies the code actually inspected.
- Setup, configuration names, decisions and known limitations are recorded; secrets and sensitive content are not committed or logged.
- A coherent Git commit leaves the repository working. Record branch/commit and distinguish local commit from pushed remote state.

Stage handoff

Return to Planning & Progress with: completed behaviour; changed files/dependencies; decisions; verification evidence; learning checkpoint; commit and push state; remaining gaps; next bounded task. No new chat is needed for every error or every day.

20 / PROVENANCE AND NEXT ACTION

Open decisions and change record

Implementation choices still to resolve

Do not invent decisions that the original sources did not settle. Resolve them at the relevant stage, record the rationale and add appropriate tests.

- Demo date, daily ApplySync hours and any explicitly reduced interim demo scope. Six stages are not a six-day estimate.
- Exact initial fields, enum values, session/query conventions and migration details; detailed event ordering, transition and supersession rules.
- Minimum required Gmail filter coverage, backfill bounds, checkpoint/reconciliation mechanics, lock strategy and independent matching-signal definitions.
- Worker library, polling cadence, retry limits, deployment provider/topology and resulting cookie/CSRF configuration.
- Current Google authorization/release requirements, Groq model/data handling, retention windows, delete-all semantics and backup handling. Verify before relying on them in production.

What changed in v2.0

Change	Effect
Errata consolidated	Connector isolation limits; refresh-token and OAuth-state rules; cookie/CSRF rules; corrected matching; filter groups; polling wording; positioning; added invariants.
Delivery reorganized	Six sequential stages replace 25 separate teaching phases; full target scope is retained.
Learning revised	Codex may implement most code; learning uses actual flows, targeted practice, debugging and review evidence.
Status made explicit	Only the reported health endpoint work is complete. No repository re-audit, deployment or resource-setting change is claimed.
Document repaired	Clean typography, numbered pages, linked contents and bookmarks; obsolete guarantees removed.

Source record

Architecture and original scope: **ApplySync_Master_Project_Reference.pdf**, v1.0, sections 1-11. Mandatory corrections: **ApplySync Errata v1.1.pdf**, items 1-10. Workflow authority: Sanjay’s explicit Planning & Progress decisions to use agent-led implementation, six stages and code-based review, consolidated on 17 September 2026. Current status: supplied Phase 1 completion handoff and reported commit 37019df. This is a consolidation, not a fresh external compliance or repository audit.

Next steps, in order

1. Add this PDF as the active reference; retain older PDFs as historical resources. 2. Replace conflicting Project instructions. 3. Create concise repository AGENTS.md guidance and make the reference available to Codex. 4. Verify Codex setup in the existing local folder. 5. Open Build 1, inspect repository/database state, and implement the next bounded task. These follow-up actions are not performed by this PDF.